The new() Function in Go

Allocating zero values and getting pointers to them

What new() does

`new()` is a built-in function that **allocates memory for a value of any type, sets it to the zero value, and returns a pointer to it**. `new(T)` gives you a `*T` pointing to a zeroed `T`.

```
p := new(int)
fmt.Println(*p) // 0
*p = 42
fmt.Println(*p) // 42
```

It is equivalent to declaring a variable and taking its address:

```
var x int
p := &x // same effect as p := new(int)
```

With structs

```
type User struct {
    Name string
    Age  int
}

u := new(User)       // *User, fields are "" and 0
u.Name = "Alice"     // Go auto-dereferences: same as (*u).Name
```

Most Go code writes `&User{}` instead, because the composite literal can also set fields in one step:

```
u := &User{Name: "Alice", Age: 30}
```

Both allocate the same way. `new(User)` and `&User{}` produce identical results.

When new() is useful

It is handy when you need a pointer to a zero value of a type that has no literal syntax for taking an address, mostly basic types:

```
count := new(int)   // you can't write &int{} or &0
flag := new(bool)
```

This comes up with optional fields in structs, for example JSON fields where `nil` means "not set" and a pointer to `0` or `false` means "explicitly set":

```
type Config struct {
    Retries *int `json:"retries,omitempty"`
}

c := Config{Retries: new(int)} // explicitly 0, not omitted
```

Generic code can also use it, since `new(T)` works for any type parameter `T`:

```
func ptrTo[T any]() *T {
    return new(T)
}
```

Where new() is a trap

For slices, maps, and channels, `new` gives you a pointer to a **nil** value, not a usable one:

```
m := new(map[string]int)
(*m)["a"] = 1 // panic: assignment to entry in nil map
```

For those three types, use `make` instead.

Stack or heap?

new does not force a heap allocation. The compiler's escape analysis decides: if the pointer never leaves the function, the value can stay on the stack. The same applies to `&T{}`. In Go you do not manage this yourself, and there is no `delete` or `free`, because the garbage collector reclaims memory when nothing references it.

Summary

	new(T)	make(T, ...)
Works with	any type	slices, maps, channels
Returns	`*T`, pointer to zero value	`T`, initialized and ready to use
Typical use	pointers to basic types, generics	creating slices, maps, channels

In everyday code, `new` is fairly rare. Structs usually use `&T{...}`, and slices, maps, and channels use `make`.